

TINY SYSTEMS PROJECT

Repeated Comment Detection System

A High-Performance Concurrent Stream Filter Written in Rust

GROUP 4

Engineering Team

26PG110058

Sandipkumar M. Bhuriya

26PG110059

Khilari Sairaj Rajendra

26PG110065

Gohil Bhavdeep J.

The High-Volume Comment Problem



Spam Bot Floods

Malicious payloads and repetitive advertisement links degrade user experience.



Exact Duplicates

Users repeatedly posting identical text threads without adding original discussion value.



Near Duplicates

Slight variations with modified punctuation or extra characters to bypass strict filters.

Concurrent Stream Pipeline

Step 01

File Ingestion

Loads comment records strictly from external dataset file (`comments.txt`).

Step 02

MPSC Channels

Simulates real-time feeds using multi-producer single-consumer thread architecture.

Step 03

Detector Engine

Evaluates incoming stream against sliding window history buffers.

Robust Text Normalization

Normalization Steps

Case Folding: Converts all alphabetic characters to lowercase.

Punctuation Stripping: Removes symbols and special characters.

Whitespace Collapse: Trims excess spaces into single word tokens.

EXAMPLE TRANSFORMATION

Raw Input:

```
"Hello, World!!! This is a Test..."
```

Normalized Output:

```
"hello world this is a test"
```

Token-Based Jaccard Scoring

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Intersection over union of word token sets.

Detection Thresholds

Exact Match: 100% normalized token equivalence.

Near Duplicate: Similarity score greater than or equal to 0.75.

Unique Comment: Score below threshold, approved for stream.

Spam Heuristics & Bot Patterns



Character Repetition

Flags payloads where consecutive repeated characters exceed 60% of total length.



Bot Payloads

Detects known malicious promotional links and free gift card scams instantly.

Early Interception

Spam heuristics are evaluated first before costly Jaccard similarity lookups.

Sliding Window History Buffer

Per-Post Tracking

Maintains a dedicated history vector grouped by unique `post\_id` parameters.

Enforces maximum capacity limits (e.g., 50 historical comments per post) to optimize memory.

FIFO Eviction

Oldest historical records are automatically evicted when buffer capacity is reached.

Ensures $O(N)$ evaluation speed per incoming comment without unbounded growth.

Live Simulation Audit Summary

100

Total Comments Processed

42

Flagged / Duplicates

58

Clean Unique Comments

GROUP 4 PRESENTATION

Thank You!

Questions & Discussion
